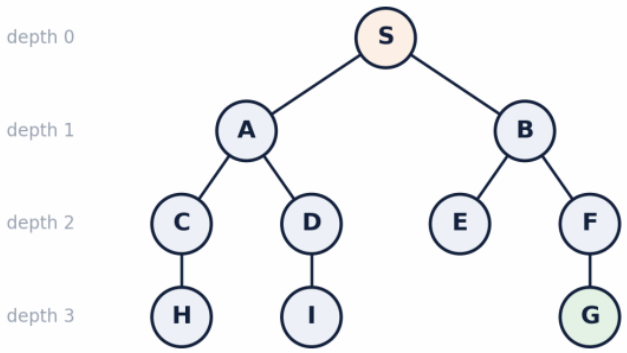
Question 5 – Depth-Limited Search (DLS)



Task: Perform Depth-Limited Search from S with depth limit ℓ = 2, looking for goal G. Explore children left to right. Complete the table.

Step	Node visited	Depth	Note
1	S	0	Expand
2	A	1	Expand
3	C	2	Cutoff
4	D	2	Cutoff
5	B	1	Expand
6	E	2	Cutoff
7	F	2	Cutoff

Questions:

1. Write the order of nodes visited.

S -> A -> C -> D -> B -> E -> F

2. Is the goal G found? Explain why.

No. G is at depth 3, beyond the given depth limit of 2.

3. What is the smallest depth limit that finds G?

The smallest depth limit that finds G is 3.

Question 9 – A* on the 8-Puzzle with Heuristics h_1 and h_2



Task: h_1 = number of misplaced tiles; h_2 = sum of Manhattan distances (do not count the blank). Run A* with h_2, moving the blank Up, Down, Left, Right, and skipping repeated states. Complete the table.

Iteration	State expanded	g	h_2	f	Children: move, h_1, h_2, f
1	123 / _46 / 758	0	3	3	Up (4,4,5); Down (4,4,5); Right (2,2,3)
2	123 / 4_6 / 758	1	2	3	Up (3,3,5); Down (1,1,3); Right (3,3,5)
3	123 / 456 / 7_8	2	1	3	Up (2,2,4); Left (2,2,4); Right (0,0,3)
4	123 / 456 / 78_	3	0	3	Goal reached
5					

Questions:

1. Compute h_1 and h_2 for the initial state.

h1 = 3 (tiles 4, 5, 8); h2 = 3 (each is one move from its goal).

2. Write the solution path and its length.

Right -> Down -> Right; solution length = 3 moves.

3. Which heuristic dominates, and why does that matter?

h2 (Manhattan distance) dominates h1 because h2 >= h1 for every state. It is more informative and admissible.

Question 11 – Simulated Annealing

Task: Simulated annealing is maximising a value. A better move is always accepted. A worse move is accepted with probability P = e^(ΔE/T), where ΔE = neighbour − current. Accept the worse move if the given random number r < P. Complete the table.

Step	Current	Neighbour	ΔE	T	P = e^(ΔE/T)	r	Accept / Reject
1	10	13	+3	20	-	—	Accept (better move)
2	13	11	-2	10	0.8187	0.55	Accept (0.55 < 0.8187)
3	11	7	-4	5	0.4493	0.62	Reject (0.62 > 0.4493)
4	11	15	+4	2	-	—	Accept (better move)
5	15	12	-3	1	0.0498	0.3	Reject (0.30 > 0.0498)

Questions:

1. Why is a worse move more likely to be accepted at high temperature?

At high temperature, the probability of accepting a worse move is higher because e^(DeltaE/T) is closer to 1.

2. What happens to simulated annealing as T approaches 0?

As T approaches 0, worse moves are almost never accepted; the algorithm behaves like hill climbing.

Question 14 – Online Depth-First Search



Task: An online agent starts at S in the maze and does not know the map. Thick lines are walls; B is a rock (blocked). At each new state the agent learns its legal actions, tried in the order Right, Down, Left, Up. Trace the online DFS agent until it reaches G.

Step	Current state	untried[s] (after choosing)	unbacktracked[s]	Action chosen
1	S	[Down]	[A]	Right -> A
2	A	[]	[S]	Backtrack left -> S
3	S	[]	[A]	Down -> C
4	C	[]	[S, D]	Right -> D
5	D	[]	[C, G]	Right -> G (goal)
6				
7				

Questions:

1. Write the path the agent actually travels.

S -> A -> S -> C -> D -> G

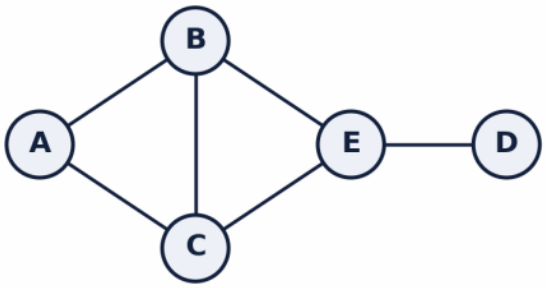
2. Why does the agent return to S after visiting A?

A has no untried route: B is a rock and the A-D passage is a wall, so the agent backtracks to S.

3. What is the competitive ratio of this run? (Optimal path: S → C → D → G.)

Actual cost = 5 moves; optimal cost = 3 moves. Competitive ratio = 5/3 = 1.67.

Question 16 – Backtracking with Forward Checking (Map Colouring)



Task: Colour regions A, B, C, D, E with Red (R), Green (G), Blue (B) so that connected regions differ. Assign variables in the order A, B, C, D, E and try values in the order R, G, B. After each assignment, apply forward checking to the neighbours' domains.

Assignment	A	B	C	D	E	Result
1	R					B,C domains become {G,B}
2	R	G				C={B}; E={R,B}
3	R	G	B			E={R}; D unrestricted
4	R	G	B	R		E domain becomes empty: failure
5	R	G	B	G		Backtrack; E={R}
6	R	G	B	G	R	Success

Questions:

1. At which assignment does forward checking detect a failure? Explain.

Failure occurs when D=Red: E has no colour left (R conflicts with D, G with B, and B with C).

2. Write the final solution.

Final solution: A=Red, B=Green, C=Blue, D=Green, E=Red.

3. How would plain backtracking (without forward checking) detect this failure?

Plain backtracking finds the failure only when it tries to colour E and all colours conflict with neighbours.